

Name: Mayank Dudhatra
UID: 24BTCSE100
Batch: B.Tech – CSE – Sem-4
Subject: Operating System

Q1. Write a C program to implement the FCFS Scheduling Algorithm

Ans:-

```
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n];

    for(i = 0; i < n; i++) {
        printf("BT of P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for(i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nP\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);

    return 0;
}
```

Q2. Write a C program to implement the SJF Scheduling Algorithm

Ans:-

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n];

    for(i = 0; i < n; i++) {
        printf("BT of P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(bt[i] > bt[j]) {
                int temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for(i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nP\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);

    return 0;
}
```

Q3. Write a C program to implement Priority Scheduling Algorithm

Ans:-

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], pr[n], wt[n], tat[n];

    for(i = 0; i < n; i++) {
        printf("BT and Priority of P%d: ", i + 1);
        scanf("%d %d", &bt[i], &pr[i]);
    }

    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(pr[i] > pr[j]) {
                int temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for(i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nP\tBT\tPR\tWT\tTAT\n");
    for(i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, bt[i], pr[i], wt[i], tat[i]);

    return 0;
}
```

Q4. Write a C program to implement Round Robin Scheduling Algorithm

Ans:-

```

#include <stdio.h>

int main() {
    int n, i, time = 0, remain, tq;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n], wt[n] = {0}, tat[n];

    for(i = 0; i < n; i++) {
        printf("BT of P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    remain = n;

    while(remain > 0) {
        for(i = 0; i < n; i++) {
            if(rt[i] > 0) {
                if(rt[i] <= tq) {
                    time += rt[i];
                    wt[i] = time - bt[i];
                    rt[i] = 0;
                    remain--;
                } else {
                    time += tq;
                    rt[i] -= tq;
                }
            }
        }
    }

    for(i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nP\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

```

```
    return 0;
}
```

Q5. Write a C program to implement First Fit Allocation Algorithm

Ans:-

```
#include <stdio.h>
```

```
int main() {
    int nb, np, i, j;
```

```
    printf("Enter number of blocks: ");
    scanf("%d", &nb);
```

```
    printf("Enter number of processes: ");
    scanf("%d", &np);
```

```
    int block[nb], process[np], alloc[np];
```

```
    for(i = 0; i < nb; i++) {
        printf("Size of block %d: ", i + 1);
        scanf("%d", &block[i]);
    }
```

```
    for(i = 0; i < np; i++) {
        printf("Size of process %d: ", i + 1);
        scanf("%d", &process[i]);
        alloc[i] = -1;
    }
```

```
    for(i = 0; i < np; i++) {
        for(j = 0; j < nb; j++) {
            if(block[j] >= process[i]) {
                alloc[i] = j;
                block[j] -= process[i];
                break;
            }
        }
    }
```

```
    printf("\nProcess\tSize\tBlock\n");
    for(i = 0; i < np; i++) {
```

```

    if(alloc[i] != -1)
        printf("P%d\t%d\t%d\n", i + 1, process[i], alloc[i] + 1);
    else
        printf("P%d\t%d\tNot Allocated\n", i + 1, process[i]);
}

return 0;
}

```

Q6. Write a C program to implement Best Fit Allocation Algorithm

Ans:-

```

#include <stdio.h>

int main() {
    int nb, np, i, j;

    printf("Enter number of blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    int block[nb], process[np], alloc[np];

    for(i = 0; i < nb; i++) {
        printf("Size of block %d: ", i + 1);
        scanf("%d", &block[i]);
    }

    for(i = 0; i < np; i++) {
        printf("Size of process %d: ", i + 1);
        scanf("%d", &process[i]);
        alloc[i] = -1;
    }

    for(i = 0; i < np; i++) {
        int best = -1;
        for(j = 0; j < nb; j++) {
            if(block[j] >= process[i]) {
                if(best == -1 || block[j] < block[best])
                    best = j;
            }
        }
    }
}

```

```

    }
}
if(best != -1) {
    alloc[i] = best;
    block[best] -= process[i];
}
}

printf("\nProcess\tSize\tBlock\n");
for(i = 0; i < np; i++) {
    if(alloc[i] != -1)
        printf("P%d\t%d\t%d\n", i + 1, process[i], alloc[i] + 1);
    else
        printf("P%d\t%d\tNot Allocated\n", i + 1, process[i]);
}

return 0;
}

```

Q7. Write a C program to implement Next Fit Allocation Algorithm

Ans:-

```

#include <stdio.h>

int main() {
    int nb, np, i;

    printf("Enter number of blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    int block[nb], process[np], alloc[np];

    for(i = 0; i < nb; i++) {
        printf("Size of block %d: ", i + 1);
        scanf("%d", &block[i]);
    }

    for(i = 0; i < np; i++) {
        printf("Size of process %d: ", i + 1);
    }
}

```

```

        scanf("%d", &process[i]);
        alloc[i] = -1;
    }

    int last = 0;

    for(i = 0; i < np; i++) {
        int count = 0;
        while(count < nb) {
            if(block[last] >= process[i]) {
                alloc[i] = last;
                block[last] -= process[i];
                break;
            }
            last = (last + 1) % nb;
            count++;
        }
    }

    printf("\nProcess\tSize\tBlock\n");
    for(i = 0; i < np; i++) {
        if(alloc[i] != -1)
            printf("P%d\t%d\t%d\n", i + 1, process[i], alloc[i] + 1);
        else
            printf("P%d\t%d\tNot Allocated\n", i + 1, process[i]);
    }

    return 0;
}

```

Q8. Write a C program to implement Worst Fit Allocation Algorithm

Ans:-

```

#include <stdio.h>

int main() {
    int nb, np, i, j;

    printf("Enter number of blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");

```

```

scanf("%d", &np);

int block[nb], process[np], alloc[np];

for(i = 0; i < nb; i++) {
    printf("Size of block %d: ", i + 1);
    scanf("%d", &block[i]);
}

for(i = 0; i < np; i++) {
    printf("Size of process %d: ", i + 1);
    scanf("%d", &process[i]);
    alloc[i] = -1;
}

for(i = 0; i < np; i++) {
    int worst = -1;
    for(j = 0; j < nb; j++) {
        if(block[j] >= process[i]) {
            if(worst == -1 || block[j] > block[worst])
                worst = j;
        }
    }
    if(worst != -1) {
        alloc[i] = worst;
        block[worst] -= process[i];
    }
}

printf("\nProcess\tSize\tBlock\n");
for(i = 0; i < np; i++) {
    if(alloc[i] != -1)
        printf("P%d\t%d\t%d\n", i + 1, process[i], alloc[i] + 1);
    else
        printf("P%d\t%d\tNot Allocated\n", i + 1, process[i]);
}

return 0;
}

```

Q9. Perform any 10 basic Linux commands

Ans:-

pwd # Show current directory
ls # List files and directories
cd .. # Go to parent directory
mkdir test # Create directory
rmdir test # Remove empty directory
touch file.txt # Create file
cp file.txt copy.txt # Copy file
mv copy.txt new.txt # Rename file
rm new.txt # Delete file
cat file.txt # View file content